

L04 · Διαίρει & Κυρίευε II

Inversions, Dominant Colour & Karatsuba

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Τρία προβλήματα που λύνονται κομψά με διαίρει και κυρίευε: μέτρηση αντιστροφών σε $O(n \log n)$, εύρεση κυρίαρχου χρώματος σε $O(n^2 \log n)$, και ο πολλαπλασιασμός Karatsuba σε $O(n^{1.585})$.

1 Τι θα δούμε

Στο προηγούμενο μάθημα είδαμε το **σχήμα** D&C και το χρησιμοποιήσαμε για ταξινόμηση. Εδώ θα δείξουμε ότι το ίδιο σχήμα — διαίρει, κυρίευσε αναδρομικά, συνδύασε — λύνει τρία εντελώς διαφορετικά προβλήματα:

1. **Μέτρηση αντιστροφών** σε πίνακα — από $O(n^2)$ πέφτουμε σε $O(n \log n)$.
2. **Κυρίαρχο χρώμα** σε σκακιέρα — από $O(n^4)$ πέφτουμε σε $O(n^2 \log n)$.
3. **Πολλαπλασιασμός μεγάλων αριθμών** — από $O(n^2)$ πέφτουμε σε $O(n^{1.585})$ με τον αλγόριθμο Karatsuba.

Σε κάθε ένα από τα τρία θα ξαναδείς το ίδιο μοτίβο: μια **έξυπνη σύνθεση** των απαντήσεων των υποπροβλημάτων που κερδίζει χρόνο. Το Master Theorem από το προηγούμενο μάθημα θα είναι το κύριο εργαλείο ανάλυσης.

Κλειδί

Το μάθημα της διάλεξης: «Διαίρει και κυρίευε» δεν είναι μια εξίσωση, είναι ένας **τρόπος σκέψης**. Όταν δεις πρόβλημα στο εξεταζόμενο μάθημα που μοιάζει άγνωστο, η πρώτη ερώτηση είναι: «μπορώ να το σπάσω σε μικρότερα ίδια προβλήματα και να συγχωνεύσω τις λύσεις σε γραμμικό χρόνο;» Αν ναι — Master Theorem.

2 Σύντομη υπενθύμιση: Master Theorem

Από το προηγούμενο μάθημα, αν

$$T(n) = a T\left(\frac{n}{b}\right) + O(n^d), \quad a > 0, b \geq 1, d \geq 0$$

τότε:

$$T(n) = \begin{cases} O(n^d) & \text{αν } d > \log_b a \\ O(n^d \log n) & \text{αν } d = \log_b a \\ O(n^{\log_b a}) & \text{αν } d < \log_b a \end{cases}$$

Διαισθητικά: βρίσκεις τη βάση b (πόσες φορές μικρότερο γίνεται το υποπρόβλημα), τον αριθμό a (πόσα τέτοια υποπροβλήματα έχεις) και το d (κόστος συγχώνευσης). Συγκρίνεις a με b^d και κερδίζει είτε η ρίζα (μεγάλο combine), είτε όλο το δέντρο εξίσου (ίσιο), είτε τα φύλλα (πολλά υποπροβλήματα).

Διαίσθηση

Σήμα κινδύνου: το Master Theorem ισχύει μόνο όταν η αναδρομή έχει σταθερό λόγο διαχωρισμού n/b . Αναδρομές όπως $T(n) = T(n/3) + T(2n/3) + n$ ή $T(n) = T(n-1) + n$ δεν λύνονται έτσι — επιστρέφουμε στις τεχνικές του προηγούμενου μαθήματος (δέντρο αναδρομής, telescoping, ισχυρή επαγωγή).

2.1 Γενικότερη μορφή (Cormen)

Στη γενικότερη διατύπωση το $O(n^d)$ αντικαθίσταται από οποιαδήποτε συνάρτηση $f(n)$:

- Αν $f(n) = O(n^{\log_b a - \epsilon})$ για κάποια $\epsilon > 0$: $T(n) = \Theta(n^{\log_b a})$.
- Αν $f(n) = \Theta(n^{\log_b a})$: $T(n) = \Theta(n^{\log_b a} \log n)$.
- Αν $f(n) = \Omega(n^{\log_b a + \epsilon})$ και $af(n/b) \leq cf(n)$ για κάποιο $c < 1$ και αρκετά μεγάλο n : $T(n) = \Theta(f(n))$.

Στο εξεταζόμενο μάθημα η απλή μορφή είναι σχεδόν πάντα αρκετή.

2.2 Ασκήσεις για να συντονιστούμε

Δοκίμασε **νοερά** να βρεις τη λύση:

- $T(n) = T(n-1) + 2$. (Όχι Master — telescope: $T(n) = \Theta(n)$.)
- $T(n) = 2T(n-1) + 1$. (Όχι Master — βγαίνει $\Theta(2^n)$, όπως ο πύργος του Hanoi.)
- $T(n) = 3T(n/4) + n$. ($a = 3$, $b = 4$, $d = 1$. $\log_4 3 \approx 0.79 < 1$, άρα **case 1**: $T(n) = O(n)$.)
- $T(n) = T(n/3) + T(2n/3) + cn$. (Όχι Master — άνισος διαχωρισμός. Δέντρο αναδρομής $\rightarrow \Theta(n \log n)$.)
- $T(n) = T(\sqrt{n}) + \log n$. (Αλλαγή μεταβλητής $n = 2^m$: $T(2^m) = T(2^{m/2}) + m$, που βγαίνει $\Theta(\log n \cdot \log \log n)$.)

3 Πρόβλημα 1: Μέτρηση Αντιστροφών

3.1 Ορισμός

Είσοδος: πίνακας A με n φυσικούς αριθμούς.

Ζητούμενο: το πλήθος των ζευγαριών (i, j) με $i < j$ και $A[i] > A[j]$.

Παράδειγμα. Στον πίνακα $[1, 3, 5, 2, 4]$ οι αντιστροφές είναι: $(3, 2)$ — το 3 είναι πριν από το 2 αλλά μεγαλύτερο — καθώς και $(5, 2)$ και $(5, 4)$. Σύνολο: **3 αντιστροφές**.

Διαίσθηση

Διαισθητικά, οι αντιστροφές μετρούν πόσο μακριά είναι ο πίνακας από το να είναι ταξινομημένος. Αν είναι ήδη ταξινομημένος $\rightarrow 0$ αντιστροφές. Αν είναι ταξινομημένος ανάποδα $\rightarrow \binom{n}{2}$ αντιστροφές (το μέγιστο).

3.2 Γιατί μας νοιάζει

Πρακτικές εφαρμογές που χρησιμοποιούν τη μέτρηση αντιστροφών ως μέτρο **ομοιότητας κατάταξης**: συνεργατικό φιλτράρισμα σε εφαρμογές μουσικής/ταινιών (πόσο μοιάζουν δύο κατατάξεις — μετράμε τις αντιστροφές μεταξύ τους), θεωρία ψηφοφορίας (απόσταση Kendall tau), μέτρηση «ποσοστού ταξινόμησης» ενός πίνακα, και ευαισθησία της σειράς κατάταξης στις μηχανές αναζήτησης.

	A	B	Γ	Δ	E
Εγώ	1	2	3	4	5
Εσύ	1	3	5	2	4

3.3 Αφελής λύση: $\Theta(n^2)$

Διπλό loop:

```
inversions(A):  
    count <- 0  
    Για i = 1 ως n-1:  
        Για j = i+1 ως n:  
            Εάν A[i] > A[j]: count <- count + 1  
    επίστρεψε count
```

Πολυπλοκότητα $\Theta(n^2)$. Για $n = 10^6$, 10^{12} συγκρίσεις $\rightarrow$ πρακτικά αδύνατο.

3.4 Λύση με D&C: $O(n \log n)$

Σπάμε τον πίνακα στη μέση και ορίζουμε **τρία είδη** αντιστροφών:

1. Αντιστροφές που είναι **εντελώς μέσα** στο αριστερό μισό A_1 .
2. Αντιστροφές που είναι **εντελώς μέσα** στο δεξί μισό A_2 .
3. Αντιστροφές **«μικτές»**: το μεγαλύτερο στοιχείο στο A_1 , το μικρότερο στο A_2 .

Τα δύο πρώτα είδη μετρώνται **αναδρομικά**. Το ζητούμενο είναι: πώς μετράμε τα μικτά γρήγορα;

Σημείωση

Παράδειγμα στον πίνακα [10, 5, 8, 6, 3, 4, 7, 1, 9, 2, 11]: το αριστερό μισό $A_1 = [10, 5, 8, 6, 3, 4]$ δίνει αναδρομικά 12 αντιστροφές, το δεξί μισό $A_2 = [7, 1, 9, 2, 11]$ δίνει 3· οι μικτές αντιστροφές είναι 15, άρα συνολικά $12 + 3 + 15 = 30$.

3.5 Πώς συνδυάζουμε σε γραμμικό χρόνο;

Παρατήρηση-κλειδί: **αν οι δύο υποπίνακες είναι ταξινομημένοι** μπορούμε να μετρήσουμε τις μικτές αντιστροφές **καθώς τους συγχωνεύουμε** σε έναν μεγαλύτερο ταξινομημένο πίνακα. Αυτό μας θυμίζει τη merge της mergesort από το προηγούμενο μάθημα — και πραγματικά, ο αλγόριθμος **ταυτόχρονα** ταξινομεί και μετράει.

Ιδέα της merge-and-count: σαρώνουμε τους δύο πίνακες με δύο δείκτες i (πάνω από A_1) και j (πάνω από A_2). Κάθε φορά παίρνουμε το μικρότερο. Αν παίρνουμε το στοιχείο από το A_2 (δηλαδή το «μπλε»

είναι μικρότερο από το «κόκκινο»), τότε **όλα τα υπόλοιπα στοιχεία του A_1 είναι μεγαλύτερα από αυτό** — άρα μεγαλύτερα **και** σε προγενέστερη θέση $\rightarrow$ όλα σχηματίζουν αντιστροφή με αυτό το στοιχείο. Προσθέτουμε στον μετρητή.

Κλειδί

Επειδή το A_1 είναι **ταξινομημένο**, αν το μπλε στοιχείο είναι μικρότερο από το τρέχον κόκκινο στη θέση i , αυτό σημαίνει ότι είναι αυτόματα μικρότερο και από όλα τα κόκκινα στις θέσεις $i, i + 1, \dots, m$. Αυτά είναι $m - i + 1$ στοιχεία. Γι' αυτό προσθέτουμε ακριβώς αυτή την ποσότητα σε μία πράξη — όχι έναν-έναν.

3.6 Ο αλγόριθμος merge-and-count

```
Διαδικασία merge-and-count(a, b):  
  // a, b ταξινομημένοι πίνακες μεγέθους m και n  
  r <- 0 // πλήθος μικτών αντιστροφών  
  i <- 1; j <- 1  
  Για k = 1 ως m + n:  
    Εάν a[i] < b[j]:  
      c[k] <- a[i]; i <- i + 1  
    αλλιώς:  
      c[k] <- b[j]; j <- j + 1  
      r <- r + (m - i + 1) // όλα τα υπόλοιπα του a είναι > b[j]  
  επιστρέψε (r, c)
```

Πολυπλοκότητα: $O(m + n)$ — μία γραμμική σάρωση. Πανομοιότυπη με την merge της mergesort, με μόνη προσθήκη τη γραμμή $r <- r + (m - i + 1)$.

3.7 Ο αλγόριθμος sort-and-count

```
Διαδικασία sort-and-count(A):  
  Εάν |A| = 1:  
    επιστρέψε (0, A)  
  Χώρισε τον A σε δύο μισά A1, A2  
  (r1, A1) <- sort-and-count(A1)  
  (r2, A2) <- sort-and-count(A2)  
  (r, A) <- merge-and-count(A1, A2)  
  επιστρέψε (r1 + r2 + r, A)
```

Επιστρέφει **και** τον αριθμό αντιστροφών **και** τον ταξινομημένο πίνακα — χρειαζόμαστε το ταξινομημένο για να μπορεί η merge-and-count του γονέα να δουλέψει.

3.8 Πολυπλοκότητα

$$T(n) = 2T(n/2) + O(n)$$

Ακριβώς η ίδια αναδρομή με την mergesort. Master Theorem: $a = 2, b = 2, d = 1, \log_2 2 = 1 = d \rightarrow$ **case 2:** $T(n) = O(n \log n)$.

Έλεγχος στο παράδειγμα. Στον πίνακα $[1, 3, 5, 2, 4]$: αριστερό $[1, 3, 5] \rightarrow 0$ αντιστροφές, ταξινομημένο· δεξί $[2, 4] \rightarrow 0$ αντιστροφές, ταξινομημένο. Συγχώνευση: παίρνω 1, παίρνω 2 (κερδίζει το δεξί ενώ

απομένουν 2 στο αριστερό από τη θέση $2 \rightarrow +2$), παίρνω 3, παίρνω 4 (κερδίζει ενώ απομένει $1 \rightarrow +1$), παίρνω 5. **Σύνολο: 3.** ✓

4 Πρόβλημα 2: Κυρίαρχο Χρώμα

4.1 Ορισμός

Είσοδος: σκακιέρα $n \times n$ (όπου $n = 2^k$) με κάθε πλακίδιο χρωματισμένο.

Ζητούμενο: να βρούμε αν υπάρχει χρώμα που εμφανίζεται σε **περισσότερα από τα μισά** πλακίδια (δηλαδή σε $> n^2/2$ θέσεις). Αν ναι, ποιο είναι.

Σημείωση

Δεν μας λένε πόσα χρώματα υπάρχουν συνολικά. Μπορεί να είναι 2, μπορεί να είναι n^2 . Αυτό κάνει το πρόβλημα μη τετριμμένο: δεν μπορούμε απλά να φτιάξουμε πίνακα μετρητών μήκους «αριθμός χρωμάτων» χωρίς να ξέρουμε πόσοι είναι.

Παραδείγματα: σκακιέρα 4×4 με 9 κόκκινα πλακίδια από 16 $\rightarrow$ κυρίαρχο χρώμα = κόκκινο ($9 > 8$). Σκακιέρα 4×4 με 8 κόκκινα $\rightarrow$ **όχι** κυρίαρχο ($8 \not> 8$ — αυστηρή ανισότητα).

4.2 Αφελής λύση: $O(n^4)$

Σαρώνουμε τα n^2 πλακίδια. Για κάθε νέο χρώμα κρατάμε μετρητή και τον ενημερώνουμε. Αλλά **σε κάθε επίσκεψη** δεν ξέρουμε αν το χρώμα είναι νέο, οπότε ψάχνουμε στη λίστα των ήδη γνωστών χρωμάτων (m χρώματα $\Rightarrow O(m)$ ανά πλακίδιο). Συνολικά $O(m \cdot n^2)$, και αφού $m \leq n^2$:

$$T(n) = O(n^4)$$

Αργό. (Με hash table πέφτει σε $O(n^2)$ αμορτοποιημένα — αλλά εδώ θέλουμε να δούμε αν το D&C βελτιώνει.)

4.3 Λύση με D&C: $O(n^2 \log n)$

Διαιρεί: σπάμε τη σκακιέρα σε 4 υποσκακιέρες $\frac{n}{2} \times \frac{n}{2}$.

Κυρίευσε: καλούμε αναδρομικά. Κάθε υποσκακιέρα επιστρέφει είτε ένα **υποψήφιο κυρίαρχο χρώμα** c_i , είτε «κανένα».

Συνδύαζε: με την εξής παρατήρηση-κλειδί:

Κλειδί

Αν η αρχική σκακιέρα έχει κυρίαρχο χρώμα c , τότε το c είναι κυρίαρχο σε **τουλάχιστον μία από τις 4 υποσκακιέρες**.

Απόδειξη: Έστω ότι το c είναι κυρίαρχο στην αρχική, άρα εμφανίζεται $> n^2/2 = 4 \cdot (n/2)^2/2$ φορές. Αν το c **δεν** ήταν κυρίαρχο σε **καμία** υποσκακιέρα, τότε σε κάθε μία εμφανίζεται **το πολύ** $(n/2)^2/2$ φορές. Άθροισμα: το πολύ $4 \cdot (n/2)^2/2 = n^2/2$ — αντίφαση. Άρα τουλάχιστον μία υποσκακιέρα έχει το c ως κυρίαρχο. ■

Αυτό κάνει τη ζωή μας εύκολη: οι μόνοι υποψήφιοι κυρίαρχοι της αρχικής σκακιέρας είναι τα (το πολύ) **4 χρώματα** που επιστρέφουν τα 4 αναδρομικά καλέσματα. Για κάθε ένα από αυτά τα 4, μετράμε σε όλη τη σκακιέρα πόσες φορές εμφανίζεται. Αυτό είναι $O(n^2)$ ανά χρώμα $\rightarrow O(4n^2) = O(n^2)$ συνολικά.

4.4 Πολυπλοκότητα

$$T(n) = 4T(n/2) + O(n^2)$$

Master Theorem: $a = 4$, $b = 2$, $d = 2$, $\log_2 4 = 2 = d \rightarrow$ **case 2**: $T(n) = O(n^2 \log n)$.

Σχόλιο: μπορεί καλύτερα; Με hash table σε $O(n^2)$ μέσο όρο. Με ντετερμινιστικό Boyer-Moore majority vote σε $O(n^2)$ ντετερμινιστικά (το πρόβλημα ανάγεται στην 1D εκδοχή). Αλλά αυτό που μετράει εδώ είναι ότι **το D&C είναι μια καθαρή, καλά αναλυμένη λύση** και είναι το ακριβές πρότυπο που η εξεταζόμενη μπορεί να ζητήσει — διότι η αρχή της λύσης (παρατήρηση + Master Theorem) μετριέται σε όλη την ύλη.

5 Πρόβλημα 3: Πολλαπλασιασμός Ακεραίων — Karatsuba

5.1 Πόσο κοστίζει ο πολλαπλασιασμός;

Δουλεύουμε με **πολύ μεγάλους αριθμούς** — π.χ. αριθμοί n ψηφίων όπου $n = 10^4$ ή 10^9 . Δεν χωράνε σε `int64`. Πρέπει να τους αναπαραστήσουμε ως πίνακες ψηφίων.

Πράξη	Πολυπλοκότητα (σχολικός τρόπος)
Πρόσθεση/πολλαπλασιασμός μονοψήφιων	$O(1)$
Πρόσθεση n -ψηφίων	$O(n)$
Πολλαπλασιασμός n -ψηφίων	$O(n^2)$

Το $O(n^2)$ προέρχεται από τη σχολική μέθοδο: για κάθε ψηφίο του πρώτου αριθμού πολλαπλασιάζουμε με όλα τα ψηφία του δεύτερου $\rightarrow n \times n$ μονοψήφιοι πολλαπλασιασμοί.

Παράδειγμα: $1357 \cdot 2468$:

$$\begin{array}{r} 1357 \\ \times 2468 \\ \hline 10856 \\ 8142 \\ 5428 \\ 2714 \\ \hline 3,349,076 \end{array}$$

5.2 Πρώτη προσπάθεια D&C — αποτυγχάνει

Σπάμε κάθε αριθμό στα μέσα ψηφία του: $1357 = 13 \cdot 10^2 + 57$ και $2468 = 24 \cdot 10^2 + 68$. Γενικά:

$$x = \alpha \cdot 10^{n/2} + \beta, \quad y = \gamma \cdot 10^{n/2} + \delta$$

όπου τα $\alpha, \beta, \gamma, \delta$ έχουν $n/2$ ψηφία. Τότε:

$$x \cdot y = \alpha\gamma \cdot 10^n + (\alpha\delta + \beta\gamma) \cdot 10^{n/2} + \beta\delta$$

Έχουμε **τέσσερις** πολλαπλασιασμούς μικρότερων αριθμών: $\alpha\gamma, \alpha\delta, \beta\gamma, \beta\delta$. Άρα $T(n) = 4T(n/2) + O(n)$. Master Theorem: $a = 4$, $b = 2$, $d = 1$, $\log_2 4 = 2 > 1 \rightarrow$ **case 3**: $T(n) = O(n^{\log_2 4}) = O(n^2)$.

Δεν κερδίσαμε τίποτα! Ίδια πολυπλοκότητα με τη σχολική μέθοδο.

5.3 Το έξυπνο τρικ του Karatsuba (1962)

Παρατηρούμε ότι θέλουμε τρεις ποσότητες: $\alpha\gamma$, $\alpha\delta + \beta\gamma$ και $\beta\delta$. Αν κάναμε **τρεις** αντί για τέσσερις πολλαπλασιασμούς, η αναδρομή θα γινόταν $T(n) = 3T(n/2) + O(n)$ που με Master Theorem δίνει $O(n^{\log_2 3}) \approx O(n^{1.585})$ — σημαντική βελτίωση.

Το κόλπο: το **μεσαίο** $\alpha\delta + \beta\gamma$ μπορεί να γραφτεί ως διαφορά τριών γινομένων που ήδη ξέρουμε:

$$(\alpha + \beta)(\gamma + \delta) = \alpha\gamma + \alpha\delta + \beta\gamma + \beta\delta$$

Άρα:

$$\alpha\delta + \beta\gamma = (\alpha + \beta)(\gamma + \delta) - \alpha\gamma - \beta\delta$$

Οι τρεις μόνο πολλαπλασιασμοί που χρειαζόμαστε είναι:

1. $z_1 = \alpha\gamma$
2. $z_2 = \beta\delta$
3. $z_3 = (\alpha + \beta)(\gamma + \delta)$

Και ο τύπος γίνεται:

$$x \cdot y = z_1 \cdot 10^n + (z_3 - z_1 - z_2) \cdot 10^{n/2} + z_2$$

Διαίσθηση

Διαισθητικά: «αγοράζουμε» μια τέταρτη πληροφορία ($\alpha\delta + \beta\gamma$) χωρίς να κάνουμε τέταρτο πολλαπλασιασμό — με μερικές προσθέσεις και αφαιρέσεις που είναι **πολύ φθηνότερες** ($O(n)$) από τους πολλαπλασιασμούς. Η ιδέα ανήκει στον Gauss για μιγαδικούς $((a + bi)(c + di))$ και την εφάρμοσε στους ακεραίους ο Karatsuba το 1962.

5.4 Ο αλγόριθμος Karatsuba

```
karatsuba(x, y):           // x, y με n ψηφία
  Εάν n = 1: επίστρεψε x * y // βάση
  m <- n/2
  Σπάσε x = α*10^m + β
  Σπάσε y = γ*10^m + δ
  z1 <- karatsuba(α, γ)
  z2 <- karatsuba(β, δ)
  z3 <- karatsuba(α + β, γ + δ)
  μέσος <- z3 - z1 - z2
  επίστρεψε z1*10^(2m) + μέσος*10^m + z2
```

Σημαντικό: το $\alpha + \beta$ και $\gamma + \delta$ μπορεί να έχουν $n/2 + 1$ ψηφία (αν κάνουν κρατούμενο), όχι ακριβώς $n/2$. Αυτή είναι μια ασήμαντη τεχνική λεπτομέρεια — δεν επηρεάζει την ασυμπτωτική ανάλυση γιατί $n/2 + 1 \leq n$.

5.5 Πολυπλοκότητα

$$T(n) = 3T(n/2) + O(n)$$

Master Theorem: $a = 3$, $b = 2$, $d = 1$. Είναι $\log_2 3 \approx 1.585 > 1$, άρα $d < \log_b a \rightarrow$ **case 3**:

$$T(n) = O(n^{\log_2 3}) = O(n^{1.585})$$

Θεώρημα (Karatsuba-Ofman, 1962). Δύο ακέραιοι n -ψηφίων μπορούν να πολλαπλασιαστούν σε $O(n^{1.585})$ βήματα.

Σχόλιο: τι ακολούθησε. Ο Karatsuba ήταν η **πρώτη** βελτίωση στο σχολικό $O(n^2)$ μετά από αιώνες. Άνοιξε ολόκληρο πεδίο έρευνας:

Έτος	Αλγόριθμος	Πολυπλοκότητα
~1450 BC	Σχολικός	$O(n^2)$
1962	Karatsuba-Ofman	$O(n^{1.585})$
1963	Toom-Cook (k -way)	$O(n^{1+1/k})$
1971	Schönhage-Strassen (FFT)	$O(n \log n \log \log n)$
2007	Fürer	$O(n \log n \cdot 2^{O(\log^* n)})$
2019	Harvey-van der Hoeven	$O(n \log n)$ ✓

Το πρόβλημα **έκλεισε** το 2019 — η εργασία Harvey-van der Hoeven έδειξε ότι ο πολλαπλασιασμός μπορεί να γίνει σε $O(n \log n)$, που έπιασε από καιρό το θεωρητικό κάτω φράγμα. Στην πράξη, για μικρότερα n το Karatsuba παραμένει το πιο αποδοτικό λόγω χαμηλών σταθερών — γι' αυτό χρησιμοποιείται σε βιβλιοθήκες όπως η GMP.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **D&C ως μέθοδος επίλυσης γενικών προβλημάτων** — όχι μόνο ταξινόμηση. Τρία διαφορετικά προβλήματα, τρεις διαφορετικές «έξυπνες συγχωνεύσεις».
2. **Μέτρηση αντιστροφών:** η merge-and-count είναι ίδια με τη merge της mergesort + μία γραμμή. Αναδρομή ίδια με mergesort $\rightarrow O(n \log n)$.
3. **Κυρίαρχο χρώμα:** η κρίσιμη παρατήρηση («αν είναι κυρίαρχο στην αρχική $\rightarrow$ είναι κυρίαρχο σε τουλάχιστον μία υποσκακιέρα») περιορίζει τους υποψήφιους σε **4**. Αυτό κάνει τη συγχώνευση γρήγορη $\rightarrow O(n^2 \log n)$.
4. **Karatsuba:** ο πρώτος αλγόριθμος που έσπασε το $O(n^2)$ για πολλαπλασιασμό. **3** αντί για **4** αναδρομικές κλήσεις, με μια αλγεβρική ταυτότητα $\rightarrow O(n^{1.585})$.
5. **Σε όλα: το Master Theorem κάνει την ανάλυση μηχανική.** Δες (a, b, d) , σύγκρινε d με $\log_b a$, διάλεξε case.